DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Ký hiệu	Tên tiếng Anh	Ý nghĩa
ACIQ	Analytical Clipping for Integer Quantization	Phương pháp chọn ngưỡng cắt bằng phân tích để lượng tử hóa số nguyên.
AI	Artificial Intelligence	Trí tuệ nhân tạo.
AP	Average Precision	Độ chính xác trung bình tổng hợp từ đường cong precision–recall.
AP50	Average Precision at IoU 0.50	AP tại ngưỡng IoU bằng 0,50.
ARM	ARM processor architecture	Kiến trúc bộ xử lý; trong đồ án sử dụng ARM Cortex-A9 của Zynq-7010.
ASIC	Application-Specific Integrated Circuit	Mạch tích hợp chuyên dụng.
AXI	Advanced eXtensible Interface	Họ giao tiếp kết nối các thành phần trong hệ thống.
AXI4	Advanced eXtensible Interface 4	Giao tiếp AXI phiên bản 4, hỗ trợ truy cập bộ nhớ theo cụm.
AXI4-Lite	AXI4-Lite interface	Biến thể AXI4 đơn giản, thường dùng cho thanh ghi điều khiển.
AXI4-Stream	AXI4-Stream interface	Giao tiếp truyền dữ liệu dạng luồng.
BGR	Blue–Green–Red	Thứ tự ba kênh màu xanh dương, xanh lá và đỏ.
BRAM	Block Random Access Memory	Bộ nhớ khối nằm trong FPGA.
CPU	Central Processing Unit	Bộ xử lý trung tâm.
DDR	Double Data Rate	Công nghệ truyền dữ liệu ở cả hai cạnh xung; dùng trong bộ nhớ DDR.
DMA	Direct Memory Access	Truy cập bộ nhớ trực tiếp.

Ký hiệu	Tên tiếng Anh	Ý nghĩa
DSP	Digital Signal Processing	Xử lý tín hiệu số; trong FPGA chỉ khối phần cứng chuyên dụng cho tính toán số học.
F1	F1-score	Trung bình điều hòa của precision và recall.
FF	Flip-Flop	Phần tử nhớ lưu một bit trạng thái.
FN	False Negative	Đối tượng chuẩn bị bỏ sót.
FP	False Positive	Phát hiện báo nhầm.
FP32	32-bit Floating Point	Biểu diễn số thực dấu phẩy động 32 bit.
FPGA	Field Programmable Gate Array	Mạch logic có thể cấu hình sau chế tạo.
FPS	Frames Per Second	Số khung hình xử lý trong một giây.
HOG	Histogram of Oriented Gradients	Đặc trưng biểu đồ hướng gradient.
II	Initiation Interval	Khoảng cách khởi tạo giữa hai lần nhận dữ liệu, tính bằng chu kỳ clock.
INT8	8-bit Integer	Biểu diễn số nguyên 8 bit.
IoU	Intersection over Union	Tỉ số diện tích giao và diện tích hợp của hai khung bao.
JPEG	Joint Photographic Experts Group	Tên nhóm tiêu chuẩn hóa, thường dùng để chỉ chuẩn nén ảnh JPEG.
JTAG	Joint Test Action Group	Tên thường dùng của giao tiếp kiểm tra và cấu hình chip.
KiB	Kibibyte	Đơn vị dung lượng bằng 1.024 byte.
LCD	Liquid Crystal Display	Màn hình tinh thể lỏng.
LUT	Look-Up Table	Bảng tra dùng để thực hiện hàm logic trong FPGA.

Ký hiệu	Tên tiếng Anh	Ý nghĩa
mAP	Mean Average Precision	Trung bình AP trên các lớp theo giao thức đánh giá.
NMS	Non-Maximum Suppression	Loại bỏ các khung không cực đại để giảm phát hiện trùng.
P95	95th Percentile	Phân vị 95%; dùng để mô tả phân bố độ trễ trong đồ án.
PC	Personal Computer	Máy tính cá nhân.
PL	Programmable Logic	Phần logic lập trình được trong SoC FPGA.
PS	Processing System	Hệ xử lý gồm CPU và các ngoại vi trong Zynq.
RGB	Red–Green–Blue	Ba kênh màu đỏ, xanh lá và xanh dương.
RGB888	8-bit Red, Green and Blue	Định dạng màu 24 bit, mỗi kênh có 8 bit.
ROI	Region of Interest	Vùng quan tâm trong ảnh.
RTL	Register Transfer Level	Mức mô tả phần cứng theo thanh ghi và luồng dữ liệu.
SoC	System on Chip	Hệ thống tích hợp trên một chip.
SPPF	Spatial Pyramid Pooling – Fast	Khối gộp đặc trưng không gian dạng kim tự tháp được dùng trong YOLOv8.
SVM	Support Vector Machine	Máy vector hỗ trợ, dùng để phân loại đặc trưng HOG.
TP	True Positive	Phát hiện đúng đối tượng theo quy tắc ghép với nhãn chuẩn.
UART	Universal Asynchronous Receiver/Transmitter	Bộ thu phát nối tiếp không đồng bộ.
YOLO	You Only Look Once	Họ mô hình phát hiện đối tượng một giai đoạn.

CHƯƠNG 1. GIỚI THIỆU CHUNG

Đồ án "Nghiên cứu hệ thống thị giác máy tính thời gian thực cho robot trên nền tảng FPGA – Edge AI" nghiên cứu bài toán phát hiện đối tượng và khả năng triển khai xử lý tại thiết bị biên. Chương này trình bày bối cảnh ứng dụng, các hướng tiếp cận, công trình liên quan, mục tiêu và phạm vi nghiên cứu. Từ đó, hai nhánh HOG–SVM và YOLOv8n được xác định làm cơ sở cho phần thiết kế và thực nghiệm.

1.1. Đặt vấn đề

1.1.1. Bối cảnh ứng dụng

Thị giác máy tính cho phép hệ thống tự động khai thác thông tin từ ảnh để xác định sự xuất hiện, loại và vị trí của đối tượng. Trong các ứng dụng robot, thông tin này có thể được sử dụng làm đầu vào cho giám sát vùng làm việc, lựa chọn vật cần thao tác hoặc hỗ trợ điều hướng. Tuy nhiên, kết quả phát hiện trên ảnh mới là một phần của hệ thống robot; để điều khiển chuyển động còn cần hiệu chuẩn camera, chuyển đổi hệ tọa độ, lập kế hoạch và kiểm tra điều kiện an toàn.

Một hệ thống thị giác thực tế cần đáp ứng đồng thời yêu cầu về chất lượng phát hiện và thời gian xử lý. Ảnh có độ phân giải cao giúp giữ chi tiết của đối tượng nhỏ nhưng làm tăng lượng dữ liệu cần tính toán và lưu trữ. Khi xử lý liên tục, chênh lệch giữa tốc độ thu nhận và tốc độ xử lý có thể tạo hàng đợi, làm thông tin đầu ra không còn phản ánh kịp trạng thái của cảnh. Vì vậy, việc lựa chọn thuật toán phải gắn với nền tảng thực thi và cách tổ chức truyền dữ liệu.

Theo hướng Edge AI, suy luận được thực hiện tại thiết bị biên thay vì phụ thuộc vào việc gửi toàn bộ ảnh lên máy chủ. SoC FPGA (System on Chip tích hợp Field Programmable Gate Array) kết hợp bộ xử lý và phần logic lập trình được, tạo điều kiện phân chia công việc giữa phần mềm và phần cứng [1]. Bộ xử lý phù hợp với điều khiển và những thao tác có luồng xử lý phức tạp; phần logic có thể thực hiện các phép toán lặp lại theo kiến trúc song song hoặc pipeline. Lợi ích chỉ được xác nhận sau khi đo trên hệ thống cụ thể, do thời gian trao đổi dữ liệu có thể làm giảm hiệu quả tăng tốc.

Đề tài tiếp cận bài toán qua hai hướng. Hướng thứ nhất sử dụng HOG–SVM để phát hiện hộp trong cảnh băng tải, thuận lợi cho việc phân tích chuỗi xử lý và mô phỏng từng khối. Hướng thứ hai sử dụng YOLOv8n với ảnh chụp từ drone để nghiên cứu mô hình học sâu và triển khai suy luận trên EBAZ4205 sử dụng Zynq-7010. Hai hướng cung cấp các trường hợp nghiên cứu về thuật toán và cách tổ chức tính toán;

chúng không được xem là một thí nghiệm so sánh độ chính xác trực tiếp khi chưa sử dụng chung dữ liệu và giao thức đánh giá.

1.1.2. Các hướng tiếp cận phát hiện đối tượng

Trong nhóm phương pháp sử dụng đặc trưng thiết kế thủ công, HOG mô tả sự phân bố hướng gradient cục bộ của ảnh. Đặc trưng này được kết hợp với SVM để phân loại một vùng ảnh. Công trình của Dalal và Triggs là cơ sở quan trọng của hướng HOG–SVM trong phát hiện người [6]. Khi chuyển sang đối tượng hộp, cần xây dựng mẫu hộp và mẫu nền phù hợp; không thể sử dụng nguyên bộ phân loại người và mặc nhiên kỳ vọng kết quả tương đương.

HOG–SVM có cấu trúc tính toán tương đối rõ: tạo vùng ứng viên, tính gradient, tích lũy histogram, chuẩn hóa và tính điểm phân loại. Cấu trúc này thuận lợi cho phân tích phần cứng. Tuy nhiên, chất lượng của bộ phát hiện còn phụ thuộc vào cách tìm vùng ứng viên. Đối tượng bị loại ở bước tạo vùng sẽ không được bộ phân loại kiểm tra, ngay cả khi bộ phân loại hoạt động tốt trên các mẫu đã cắt đúng.

Đối với học sâu, YOLO tổ chức phát hiện đối tượng thông qua một mạng nơ-ron dự đoán vị trí và lớp đối tượng từ ảnh đầu vào [15]. YOLOv8 sử dụng đầu phát hiện không dựa trên các hộp neo định sẵn và tách các nhánh dự đoán trong đầu ra [18]. Đề tài sử dụng biến thể YOLOv8n để khảo sát triển khai trên nền tảng có tài nguyên hạn chế. Kích thước mô hình nhỏ hơn không tự bảo đảm tốc độ thời gian thực, bởi tốc độ còn phụ thuộc kiểu số, bộ nhớ, phần mềm thực thi và mức độ sử dụng bộ tăng tốc.

1.1.3. Tổng quan công trình liên quan

Các công trình được tổng hợp theo nền tảng, kết quả công bố và vai trò đối với đồ án. Việc đối chiếu phải xét đồng thời độ phân giải, số tỉ lệ phát hiện, loại đối tượng, kiểu dữ liệu và phạm vi phép đo. Một kết quả trên ASIC hoặc FPGA lớn không phải là cam kết hiệu năng cho Zynq-7010.

Bảng 1.1. Các công trình liên quan và vai trò đối với đồ án

Công trình	**Nền tảng**	**Nội dung hoặc kết quả công bố**	**Vai trò trong đồ án**
Dalal và Triggs, 2005 [6]	Phần mềm	HOG kết hợp SVM cho phát hiện người.	Cơ sở đặc trưng gradient và bộ phân loại.

Công trình	Nền tảng	Nội dung hoặc kết quả công bố	Vai trò trong đồ án
Hahnle và cộng sự, 2013 [8]	FPGA	Phát hiện người đa tỉ lệ trên ảnh 1920 × 1080; công bố 64 FPS ở 18 tỉ lệ.	Tham khảo tổ chức song song và chia sẻ phần cứng theo thời gian.
Mizuno và cộng sự, 2012 [11]	FPGA	Nghiên cứu kiến trúc bộ xử lý trích đặc trưng HOG.	Tham khảo pipeline theo cell và tổ chức bộ nhớ.
Suleiman và Sze, 2014 [17]	ASIC	Thiết kế HOG đa tỉ lệ cho 1080HD, 60 FPS.	Tham khảo hiệu quả năng lượng; không so trực tiếp với FPGA.
Luo và Lin, 2018 [10]	Cyclone IV	800 × 600; khoảng 162 FPS tính từ chu kỳ; 16.060 LUT, 334 Kbit bộ nhớ.	Tham khảo HOG–SVM dạng luồng và tái sử dụng dữ liệu.
Wasala và Kryjak, 2022 [22]	ZCU104	HOG–SVM 3840 × 2160, 60 FPS, một tỉ lệ.	Tham chiếu khả năng xử lý luồng trên SoC FPGA lớn hơn.
FINN, 2017 [20] và FINN-R, 2018 [4]	FPGA / SoC FPGA	Sinh kiến trúc phần cứng cho mạng nhị phân hoặc lượng tử hóa.	Cơ sở tham khảo về kiểu số thấp và ánh xạ mạng lên phần cứng.
Nguyễn Trung Kiên, 2024 [9]	PC / C++	Zipfian Estimation, HOG–SVM đa tỉ lệ; đánh giá trên video MOT15.	Tham khảo luồng dữ liệu, lựa chọn vùng và cách đánh giá; chưa phải thực nghiệm trên bo.

Các nghiên cứu HOG trên FPGA cho thấy việc tái sử dụng dữ liệu giữa các cell, block và cửa sổ có ý nghĩa lớn đối với bộ nhớ và thông lượng. Đặc biệt, Luo và Lin công bố kiến trúc HOG–SVM hoàn toàn trên Cyclone IV, không dùng bộ nhớ ngoài; tốc độ khoảng 162 khung/s được suy ra từ số chu kỳ và tần số 150 MHz [10]. Do sự

khác nhau về cấu trúc LUT, bộ nhớ và mức song song giữa Cyclone IV và Zynq-7010, khả năng áp dụng kiến trúc cần được xác định bằng tổng hợp trên thiết bị mục tiêu.

FINN và FINN-R cho thấy lượng tử hóa có thể được kết hợp với tổ chức phần cứng chuyên dụng để thực thi mạng nơ-ron trên FPGA [20], [4]. Ý nghĩa đối với đề tài là cần nghiên cứu đồng thời biểu diễn số, bộ nhớ và phân chia phép tính. Trong hướng triển khai của đồ án, lượng tử hóa được khảo sát cùng với phân chia công việc giữa ARM và FPGA; đây là cách tổ chức khác với việc sinh một kiến trúc FINN cho toàn mạng.

Luận văn của Nguyễn Trung Kiên nghiên cứu chuỗi phát hiện đối tượng gồm chọn vùng chuyển động bằng Zipfian Estimation, xử lý HOG–SVM đa tỉ lệ và loại khung trùng [9]. Thực nghiệm của luận văn được thực hiện bằng C++ trên máy tính với video MOT15; hướng FPGA được đặt ra như mục tiêu triển khai tiếp theo. Công trình có ý nghĩa tham khảo về cách phân chia hệ thống, mô tả luồng dữ liệu và đánh giá đồng thời chất lượng phát hiện với tốc độ xử lý. Trong đồ án này, nhánh hộp sử dụng đặc điểm nền và màu để tạo vùng ứng viên; nhánh YOLOv8n sử dụng mạng nơ-ron để dự đoán trực tiếp. Vì vậy, thuật toán Zipfian Estimation được xem là một hướng nghiên cứu liên quan, không phải thành phần đã cài đặt trong hai nhánh đang xét.

1.1.4. Vấn đề nghiên cứu

Vấn đề thứ nhất là bảo đảm thuật toán phù hợp với dữ liệu. Cảnh băng tải có camera cố định cho phép khai thác vùng quan tâm và đặc điểm nền; ảnh drone có góc nhìn, tỉ lệ đối tượng và mức che khuất đa dạng hơn. Một cấu hình hiệu quả trên cảnh thứ nhất chưa chứng minh khả năng tổng quát hóa sang cảnh thứ hai.

Vấn đề thứ hai là chuyển mô hình tính toán sang phần cứng mà vẫn kiểm soát được sai số. Những phép toán dùng số thực trên PC có thể được thay bằng số nguyên hoặc số cố định trên FPGA. Việc thay đổi này ảnh hưởng đến điểm phân loại, tọa độ và thứ tự các dự đoán. Do đó, phải có kết quả tham chiếu và kiểm tra sai số ở từng mức, từ phép toán cơ bản đến đầu ra của bộ phát hiện.

Vấn đề thứ ba là giới hạn tài nguyên và chi phí trao đổi dữ liệu. Bộ tăng tốc có thể xử lý nhanh một phép nhân–cộng nhưng toàn hệ thống vẫn chậm nếu phần lớn phép tính còn chạy trên CPU hoặc dữ liệu được truyền không hiệu quả. Đồ án vì vậy xem xét cả chất lượng phát hiện, độ trễ, thông lượng và tài nguyên, thay vì chỉ dựa vào tần số clock của FPGA.

1.2. Mục tiêu và phạm vi nghiên cứu

1.2.1. Mục tiêu nghiên cứu

Mục tiêu tổng quát là xây dựng và đánh giá các thành phần của hệ thống thị giác phát hiện đối tượng theo hướng Edge AI trên SoC FPGA, làm cơ sở cho ứng dụng robot có yêu cầu đáp ứng theo thời gian. Nghiên cứu được tiến hành từ chương trình tham chiếu trên PC, mô phỏng RTL đến thử nghiệm suy luận trên bo. Mức độ đáp ứng thời gian thực được đánh giá thông qua độ trễ và thông lượng của cấu hình thực nghiệm.

Các mục tiêu cụ thể gồm: nghiên cứu HOG–SVM và YOLOv8n; xác định dữ liệu và giao thức đánh giá cho từng nhánh; xây dựng hoặc sử dụng mô hình phù hợp với lớp đối tượng; kiểm chứng các khối xử lý trên Vivado; triển khai suy luận có phân chia công việc giữa ARM và FPGA; đo kết quả trên đúng cấu hình được sử dụng và chỉ ra giới hạn khi mở rộng hệ thống.

Ở mức đầu ra, hệ thống cần cung cấp danh sách đối tượng gồm lớp, tọa độ khung bao và điểm dự đoán. Ảnh vẽ khung phục vụ quan sát kết quả, còn số liệu đánh giá phải được tính từ dự đoán và nhãn chuẩn. Chức năng hiển thị được kiểm tra riêng với chất lượng phát hiện nhằm phân biệt kết quả của khối xử lý và đường xuất ảnh.

1.2.2. Dữ liệu và đối tượng nghiên cứu

Nhánh HOG–SVM sử dụng bộ Cement Conveyor Box Detection trên Roboflow, được lưu trong dự án với tên BOX_V4 [16]. Đây là bài toán một lớp hộp trong cảnh băng tải. Ảnh toàn cảnh và các vùng ảnh chuẩn hóa 64 × 64 pixel phục vụ hai vai trò khác nhau: ảnh toàn cảnh dùng để đánh giá phát hiện, còn vùng chuẩn hóa dùng để trích đặc trưng HOG và phân loại. Tên BOX_V4 được dùng để quản lý dữ liệu và các sản phẩm thí nghiệm trong dự án.

Nhánh YOLOv8n sử dụng ảnh từ VisDrone, một nguồn dữ liệu phục vụ phát hiện và theo dõi đối tượng từ góc nhìn trên không [21]. Cấu hình của dự án tập trung vào năm lớp phương tiện: car, van, truck, bus và motor. Phạm vi này là một tập con các lớp của VisDrone. Các thí nghiệm sử dụng ảnh đã thu nhận; việc tích hợp camera và lắp đặt hệ thống trên drone không thuộc phạm vi kiểm chứng hiện tại.

Hai nguồn dữ liệu được quản lý và đánh giá riêng. Khi cần so sánh HOG–SVM với YOLO về độ chính xác, phải thiết kế thêm thí nghiệm chung về lớp đối tượng, ảnh kiểm thử, nhãn và quy tắc ghép khung. Trong phạm vi hiện tại, bảng đối chiếu giữa hai nhánh chủ yếu làm rõ đặc điểm phương pháp, điều kiện sử dụng và yêu cầu triển khai.

1.2.3. Phạm vi phần mềm và phần cứng

Bảng 1.2. Phạm vi của hai nhánh nghiên cứu

Nội dung	**HOG–SVM**	**YOLOv8n**
Dữ liệu	Cement Conveyor Box Detection, tên nội bộ BOX_V4.	VisDrone, phạm vi năm lớp phương tiện.
Đối tượng	Hộp trong cảnh băng tải.	Car, van, truck, bus, motor.
Đầu vào	Ảnh toàn cảnh; vùng ứng viên chuẩn hóa để phân loại.	Ảnh được tiền xử lý theo cấu hình mạng; bản khảo sát bo dùng 320 × 320.
Hướng triển khai	Chương trình PC và nghiên cứu các khối RTL trong Vivado.	Chương trình tham chiếu và suy luận phối hợp ARM–FPGA trên EBAZ4205.
Đầu ra	Khung bao, điểm SVM và ảnh minh họa.	Khung bao, lớp, điểm dự đoán và ảnh minh họa.

Nền tảng phần cứng được sử dụng là EBAZ4205 với thiết bị xc7z010clg400-1, thay cho nền tảng KV260 dự kiến trong báo cáo ban đầu. Zynq-7010 có hệ xử lý ARM và vùng logic FPGA trong cùng chip [1]. Bản suy luận YOLO được tổ chức theo hướng phối hợp PS–PL: phần mềm ARM điều khiển luồng thực thi và phối hợp với lõi tính toán trong FPGA.

Phạm vi thực nghiệm tập trung vào phát hiện đối tượng, kiểm chứng các khối xử lý và khảo sát suy luận trên bo. Xử lý camera liên tục, định vị trong hệ tọa độ robot và điều khiển robot khép kín là hướng tích hợp tiếp theo, phụ thuộc vào độ trễ, giao tiếp và độ tin cậy của bộ phát hiện.

1.2.4. Chỉ tiêu đánh giá

Bảng 1.3. Các nhóm chỉ tiêu đánh giá

Nhóm	**Chỉ tiêu**	**Ý nghĩa và điều kiện**
Phát hiện	TP, FP, FN; precision, recall, F1	Tính từ dự đoán và nhãn chuẩn tại ngưỡng đã xác định.

Nhóm	Chỉ tiêu	Ý nghĩa và điều kiện
Chất lượng tổng hợp	AP50; AP@[0,50:0,95] hoặc mAP tương ứng	Ghi rõ lớp, tập ảnh và giao thức đánh giá.
Kiểm chứng số học	Số đầu ra sai khác; sai số tuyệt đối	Đối chiếu cùng dữ liệu với mô hình tham chiếu.
Hiệu năng	Thời gian mỗi ảnh, FPS, phân bố độ trễ	Nêu khâu được đo, kích thước ảnh và cấu hình thực thi.
Phần cứng	LUT, FF, BRAM, DSP; timing	Lấy từ báo cáo đúng thiết bị và phiên bản thiết kế.

Các ngưỡng đạt yêu cầu phải gắn với từng thí nghiệm và ứng dụng. Precision, recall và AP phản ánh các khía cạnh khác nhau của chất lượng phát hiện và được báo cáo riêng. Tương tự, thời gian mô phỏng trên máy tính, số chu kỳ của RTL và thời gian đo trên bo là ba đại lượng khác nhau. Phương pháp tính các chỉ tiêu được trình bày ở Chương 2; cấu hình và số liệu thực nghiệm được trình bày ở Chương 4.

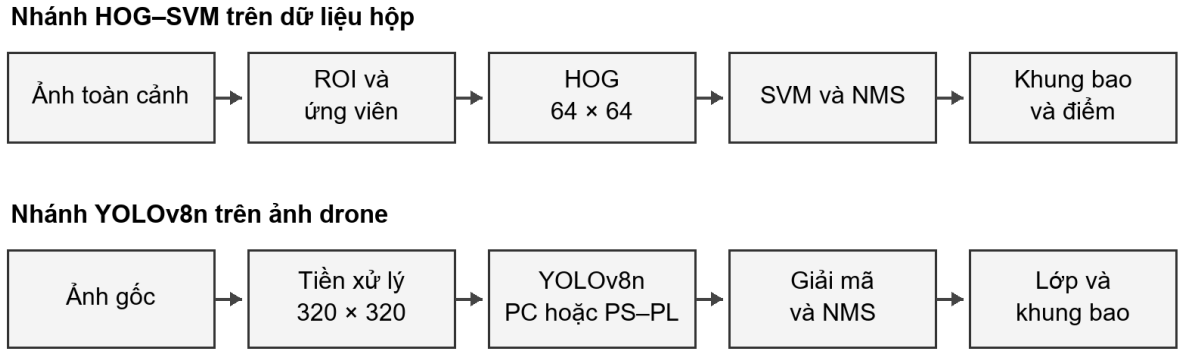
1.3. Phương pháp nghiên cứu và giải pháp hệ thống

1.3.1. Quy trình nghiên cứu

Quy trình gồm xác định đầu vào và nhãn chuẩn, xây dựng chương trình tham chiếu, kiểm tra thuật toán trên dữ liệu cố định, mô tả các khối phần cứng, đối chiếu mô phỏng và thử nghiệm trên bo. Mỗi kết quả được gắn với phiên bản mô hình, cấu hình tiền xử lý và điều kiện đo. Khi thay đổi kiểu số hoặc cách phân chia công việc, cần kiểm tra lại đầu ra trước khi kết luận về tốc độ.

Tập huấn luyện được dùng để học tham số mô hình; tập validation phục vụ chọn cấu hình; tập test được giữ độc lập với các quyết định điều chỉnh. Với lượng tử hóa sau huấn luyện, tập calibration được dùng để xác định miền giá trị số. Tập test được giữ tách biệt với quá trình điều chỉnh ngưỡng, lựa chọn phương pháp lượng tử hóa và bổ sung mẫu huấn luyện.

1.3.2. Kiến trúc xử lý của hai nhánh



Hình 1.1. Sơ đồ xử lý của hai nhánh nghiên cứu trong đồ án

Với HOG–SVM, ảnh được đưa qua bước chọn vùng xử lý và tạo ứng viên. Mỗi ứng viên được chuẩn hóa kích thước, trích đặc trưng HOG và phân loại bằng SVM; các khung trùng được lọc trước khi xuất kết quả. Các mô-đun RTL được kiểm chứng bằng dữ liệu pixel và kết quả tham chiếu tương ứng. Nếu mô hình phần cứng có xấp xỉ hoặc cách tạo ứng viên khác với PC, sự khác biệt phải được ghi rõ khi đánh giá.

Với YOLOv8n, ảnh được tiền xử lý về kích thước đầu vào của mạng, suy luận để tạo dự đoán, sau đó giải mã, loại khung trùng và chuyển tọa độ về ảnh gốc. Trên Zynq, ARM điều khiển quá trình và thực hiện phần mềm, còn lõi PL đảm nhiệm các phép tính được phân công. Đầu ra sau hậu xử lý được sử dụng để vẽ khung và phục vụ hiển thị trên PC hoặc LCD. Trong hệ thống tích hợp, mỗi ảnh hiển thị cần được liên kết với kết quả suy luận tương ứng để kiểm tra tính nhất quán của luồng dữ liệu.

1.3.3. Cách kiểm chứng và ý nghĩa dự kiến

Kiểm chứng được thực hiện theo ba mức: tính đúng của phép toán, tính đúng của chuỗi xử lý và chất lượng phát hiện. Dạng sóng giúp xác định thời điểm dữ liệu hợp lệ và sự phối hợp giữa các khối. Đối chiếu số học giúp phát hiện lỗi dấu, tràn số và thứ tự dữ liệu. Đánh giá với nhãn chuẩn mới cho biết đối tượng được phát hiện đúng, báo nhầm hoặc bỏ sót.

Đóng góp hướng tới của đồ án là một quy trình có thể kiểm tra lại từ thuật toán đến phần cứng, cùng phân tích những giới hạn khi đưa thị giác máy tính lên một SoC FPGA tài nguyên nhỏ. Các kết quả giúp xác định phần tính toán nên tăng tốc, các chi phí còn chi phối thời gian xử lý và điều kiện cần bổ sung trước khi phục vụ ứng dụng robot.

1.4. Bố cục đồ án

Chương 1 trình bày bối cảnh, tổng quan công trình, vấn đề nghiên cứu, mục tiêu, phạm vi và giải pháp tổng thể. Chương 2 trình bày cơ sở lý thuyết về ảnh số, HOG–SVM, YOLOv8n, lượng tử hóa, SoC FPGA, Edge AI và các phương pháp đánh giá.

Chương 3 trình bày thiết kế và xây dựng hệ thống: chuẩn bị dữ liệu, cấu hình thuật toán, mô-đun RTL, phân chia PS–PL và giao tiếp đầu ra. Chương 4 trình bày điều kiện thực nghiệm, kết quả trên PC, mô phỏng Vivado và thử nghiệm trên bo, đồng thời phân tích các giới hạn. Chương 5 tổng kết kết quả, mức độ đáp ứng mục tiêu và hướng phát triển.

1.5. Kết luận chương

Chương 1 đã xác định bài toán phát hiện đối tượng trong điều kiện hạn chế tài nguyên của thiết bị biên và làm rõ vai trò của SoC FPGA. Các công trình liên quan cho thấy chất lượng phát hiện, tổ chức bộ nhớ và mức độ song song cần được xem xét đồng thời. Trên cơ sở đó, đồ án lựa chọn hai nhánh nghiên cứu: HOG–SVM cho cảnh băng tải và YOLOv8n cho ảnh phương tiện từ góc nhìn trên không. Cơ sở lý thuyết của hai nhánh cùng phương pháp đánh giá được trình bày trong Chương 2.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày nền tảng của hai phương pháp HOG–SVM và YOLOv8n, cùng các khái niệm cần thiết để triển khai trên SoC FPGA. Nội dung được tổ chức từ biểu diễn ảnh, tạo đặc trưng và dự đoán đến lượng tử hóa, giao tiếp phần cứng và đánh giá kết quả. Các công thức mô tả nguyên lý; cấu hình cụ thể và bằng chứng thực nghiệm được trình bày trong các chương thiết kế và đánh giá.

2.1. Thị giác máy tính và bài toán phát hiện đối tượng

2.1.1. Phân loại, phát hiện và theo dõi

Phân loại ảnh xác định lớp của một ảnh hoặc vùng ảnh đã cho. Phát hiện đối tượng còn phải xác định vị trí và kích thước của từng đối tượng trong ảnh. Theo dõi đối tượng bổ sung việc liên kết các phát hiện giữa nhiều khung hình để duy trì định danh theo thời gian. Vì vậy, việc phát hiện được một hộp hoặc phương tiện trong một ảnh chưa đồng nghĩa hệ thống đã theo dõi chuyển động hoặc điều khiển được robot.

Đầu vào của bộ phát hiện là ảnh có kích thước W × H. Đầu ra là một tập các dự đoán; mỗi dự đoán gồm khung bao, lớp và điểm đánh giá. Có thể biểu diễn khung bao bằng góc trên trái (x, y), chiều rộng w và chiều cao h, hoặc bằng hai góc (x_1, y_1) và (x_2, y_2). Quy ước tọa độ phải được thống nhất giữa nhãn, tiền xử lý, bộ phát hiện và chương trình vẽ khung.

2.1.2. Thông tin đầu ra phục vụ robot

Tọa độ khung bao thuộc hệ tọa độ ảnh và có đơn vị pixel. Tâm khung có thể dùng làm thông tin quan sát ban đầu, nhưng không trực tiếp cho biết vị trí theo đơn vị milimét trong không gian robot. Để chuyển sang tọa độ vật lý cần mô hình camera, thông số hiệu chuẩn và giả thiết về mặt phẳng hoặc thông tin độ sâu. Đồ án xem đầu ra phát hiện là dữ liệu cho khối điều khiển trong hướng phát triển, chưa thay thế toàn bộ quá trình định vị và điều khiển.

2.2. Biểu diễn ảnh và tiền xử lý

2.2.1. Ảnh màu, ảnh xám và bộ nhớ

Ảnh RGB 8 bit mỗi kênh dùng ba giá trị nguyên trong khoảng 0–255 cho một pixel. Ảnh xám 8 bit dùng một giá trị để biểu diễn cường độ. Phép chuyển đổi thường dùng trong OpenCV có dạng [12]:

$$I = 0{,}299R + 0{,}587G + 0{,}114B \quad (2.1)$$

Trong đó R, G và B là ba kênh màu; kết quả được làm tròn và giới hạn về miền biểu diễn. Thứ tự kênh BGR của dữ liệu OpenCV cần được phân biệt với RGB của mô hình hoặc mạch. Sai thứ tự kênh có thể làm thay đổi cả màu hiển thị lẫn đầu vào suy luận.

Lượng bộ nhớ ảnh thô được xác định bởi số pixel và số bit cho mỗi pixel. Ví dụ tính toán, ảnh 512 × 512 RGB888 cần 786.432 byte, tương đương 768 KiB; ảnh xám cùng kích thước cần 256 KiB. Đây là dung lượng dữ liệu ảnh chưa nén, không phải dung lượng tệp JPEG. Ví dụ này cho thấy bộ đệm toàn ảnh có thể chiếm phần lớn hoặc vượt bộ nhớ trong FPGA nhỏ, nên cần cân nhắc bộ đệm dòng, DDR hoặc xử lý theo từng phần.

2.2.2. Vùng quan tâm và thay đổi kích thước

Vùng quan tâm, gọi là ROI, giới hạn phần ảnh cần xử lý. ROI cố định phù hợp khi camera và vùng làm việc ít thay đổi, nhưng có thể làm bỏ sót đối tượng nằm ngoài vùng đã chọn. Tọa độ của kết quả phát hiện trong ROI phải được cộng lại độ lệch của ROI để biểu diễn trên ảnh gốc.

Trong nhánh HOG–SVM, mỗi ứng viên được đưa về 64 × 64 pixel để tạo vector đặc trưng có cùng số chiều với mẫu huấn luyện. Đây là kích thước vùng phân loại, không phải yêu cầu chia ảnh toàn cảnh thành các ô không chồng lấn. Cách cắt cứng có thể làm đối tượng nằm trên ranh giới bị chia đôi; tạo ứng viên hoặc quét cửa sổ chồng lấn giúp xét các vị trí phù hợp hơn.

Trong nhánh YOLO, đầu vào mạng có kích thước cố định theo cấu hình thử nghiệm. Khi dùng letterbox, ảnh được co giãn giữ tỉ lệ rồi bổ sung phần đệm. Với kích thước đích $W_n \times H_n$, hệ số co giãn lý tưởng là:

$$r = min(\frac{W_n}{W}, \frac{H_n}{H}) \quad (2.2)$$

Nếu phần đệm ở bên trái và phía trên lần lượt là p_x và p_γ, tọa độ dự đoán trong ảnh mạng được chuyển về ảnh gốc theo $x = (x' - p_x)/r$ và $y = (y' - p_\gamma)/r$. Khi cài đặt phải dùng đúng kích thước đã làm tròn và phần đệm thực tế, sau đó giới hạn tọa độ trong ảnh. Cách co giãn, giá trị đệm và chuẩn hóa kênh phải giống quy trình dùng khi chuẩn bị mô hình.

2.3. Đặc trưng HOG

2.3.1. Tính gradient

HOG biểu diễn hình dạng thông qua sự phân bố hướng biến thiên cường độ trong các vùng cục bộ [6]. Với ảnh xám I, một cách tính gradient là sai phân trung tâm theo hai trục:

$$G_x(x,y) = I(x+1,y) - I(x-1,y)$$
$$G_\gamma(x,y) = I(x,y+1) - I(x,y-1) \quad (2.3)$$

Độ lớn g phản ánh mức biến thiên, còn góc θ phản ánh hướng của gradient:

$$g = \sqrt{G_x{}^2 + G_\gamma{}^2}; \quad \theta = atan2(G_\gamma, G_x)\ mod\ \pi \quad (2.4)$$

Trong HOG không dấu, các hướng đối nhau được gộp vào khoảng từ 0 đến dưới 180°. Khi triển khai, cần quy định cách xử lý pixel ở biên ảnh, quy tắc tính góc và độ rộng dữ liệu. Các lựa chọn này ảnh hưởng đến khả năng đối chiếu giữa mô hình tham chiếu và RTL.

2.3.2. Histogram theo cell và chuẩn hóa block

Ảnh được chia thành các cell. Mỗi pixel đóng góp vào histogram hướng của cell theo trọng số liên quan đến độ lớn gradient. Nếu góc nằm giữa hai khoảng hướng, có thể phân bố đóng góp cho hai khoảng lân cận. Cách nội suy theo hướng và vị trí phải nhất quán giữa quá trình huấn luyện và suy luận; chỉ giữ khoảng hướng gần nhất là một phép xấp xỉ cần đánh giá riêng.

Nhiều cell lân cận tạo thành block. Chuẩn hóa block giúp giảm tác động của sự thay đổi độ tương phản. Với vector histogram h và hằng số nhỏ ε, chuẩn hóa L2 được biểu diễn bởi:

$$v = \frac{h}{\sqrt{\|h\|_2{}^2 + \varepsilon^2}} \quad (2.5)$$

L2-Hys tiếp tục giới hạn các thành phần đã chuẩn hóa theo một mức cắt, rồi chuẩn hóa lại. Sau đó, các vector block được ghép theo thứ tự xác định thành vector HOG. OpenCV cung cấp các tham số cửa sổ, block, bước dịch block, cell và số khoảng hướng để cấu hình bộ mô tả [13].

2.3.3. Kích thước vector trong nhánh phát hiện hộp

Với cửa sổ 64 × 64, block 16 × 16, bước dịch block 8 × 8, cell 8 × 8 và 9 khoảng hướng, có 7 block trên mỗi chiều. Mỗi block gồm 4 cell; số chiều đặc trưng được tính như sau:

$$D = (\frac{64 - 16}{8} + 1) \times (\frac{64 - 16}{8} + 1) \times 4 \times 9 = 1764 \quad (2.6)$$

Con số 1.764 là số thành phần của vector dùng cho một vùng ảnh, không phải số đối tượng hay số pixel đầu vào. Toàn bộ các thành phần phải được đưa vào đúng thứ tự khi nhân với trọng số SVM. Nếu thay đổi kích thước cửa sổ hoặc tham số HOG, mô hình SVM phải tương thích với vector mới.

2.3.4. Tổ chức tính HOG trên FPGA

Các phép sai phân và tích lũy histogram có thể được tổ chức thành các tầng pipeline. Bộ đệm dòng cung cấp các pixel lân cận; bộ nhớ histogram lưu tổng theo hướng; bộ chuẩn hóa tạo vector block và chuyển sang bộ phân loại. Những block chồng lấn có thể tái sử dụng thông tin cell để giảm tính toán lặp.

Phép căn bậc hai, chia và tính góc cần được lựa chọn cách cài đặt phù hợp. Có thể sử dụng bảng tra, thuật toán lặp hoặc xấp xỉ, nhưng việc thay đổi công thức tham chiếu có thể làm giảm chất lượng đặc trưng. Do đó cần kiểm tra riêng sai số số học và ảnh hưởng đến phát hiện, thay vì chỉ kiểm tra mạch có tạo được đầu ra.

2.4. Bộ phân loại SVM tuyến tính

2.4.1. Hàm quyết định

SVM là phương pháp học có giám sát nhằm xây dựng biên phân chia giữa các lớp [5]. Trong nhánh phát hiện hộp, mỗi mẫu là một vector HOG z cùng nhãn hộp hoặc nền. Hàm quyết định của SVM tuyến tính có dạng:

$$s(z) = w^T z + b \quad (2.7)$$

Trong đó w là vector trọng số và b là hệ số tự do. Khi quy ước chiều điểm tăng tương ứng với lớp hộp, ứng viên được giữ nếu s(z) lớn hơn hoặc bằng ngưỡng τ. Giá trị s(z) là điểm của hàm quyết định. Khoảng cách có dấu đến siêu phẳng phân chia là $s(z)/\|w\|_2$ khi w khác 0; cả hai đều không mặc nhiên là xác suất. Quy ước dấu của thư viện được xác định theo nhãn lớp trước khi áp dụng ngưỡng.

2.4.2. Huấn luyện biên mềm

Với các mẫu (z_i, y_i), trong đó y_i thuộc $\{-1, +1\}$, bài toán biên mềm cân bằng giữa độ lớn vector trọng số và mức vi phạm biên. Dạng tối ưu thường dùng là:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{N} \xi_i \quad (2.8)$$

Với các ràng buộc:

$$y_i(w^T z_i + b) \geq 1 - \xi_i; \quad \xi_i \geq 0 \quad (2.9)$$

Tham số C điều chỉnh mức phạt đối với các mẫu vi phạm biên, còn ξ_i cho phép xử lý dữ liệu không phân tách hoàn toàn. C và ngưỡng quyết định cần được lựa chọn theo dữ liệu huấn luyện hoặc validation theo giao thức đã xác định. Mô hình chỉ lưu những tham số cần thiết cho suy luận; không cần thực hiện lại quá trình huấn luyện trên FPGA.

2.4.3. Mẫu âm khó và suy luận phần cứng

Mẫu âm khó là vùng nền bị bộ phát hiện nhận nhầm thành hộp. Quy trình bổ sung mẫu âm khó thực hiện bộ phát hiện trên phần dữ liệu huấn luyện, chọn các vùng nhận nhầm đã kiểm tra nhãn rồi huấn luyện lại. Dữ liệu cho bước này được lấy từ tập huấn luyện và giữ tách biệt với tập test. Các mẫu cắt từ cùng một ảnh cũng không được tính là những ảnh độc lập khi mô tả quy mô dữ liệu.

Sau huấn luyện, suy luận SVM tuyến tính chủ yếu là chuỗi nhân–cộng và so sánh ngưỡng. Một lõi phần cứng có thể tính nối tiếp để tiết kiệm tài nguyên hoặc dùng

nhiều bộ nhân–cộng song song để tăng thông lượng. Bộ tích lũy cần đủ độ rộng; trọng số lượng tử hóa và đặc trưng phải được đặt cùng quy ước tỉ lệ trước khi cộng hệ số b.

2.5. Tạo vùng ứng viên và hậu xử lý

2.5.1. Tạo ứng viên từ ảnh hoặc chuyển động

Cửa sổ trượt kiểm tra nhiều vị trí và tỉ lệ trên ảnh. Số cửa sổ tăng khi giảm bước dịch hoặc bổ sung nhiều tỉ lệ, kéo theo số lần tính đặc trưng và phân loại. Tạo vùng ứng viên bằng nền hoặc màu có thể giảm số vùng phải kiểm tra trong cảnh có điều kiện ổn định.

Với ảnh hiện tại I_t và ảnh nền tham chiếu B, độ khác nền có thể tính theo từng pixel. Nếu thay B bằng khung I_{t-1}, phép toán trở thành chênh lệch hai khung liên tiếp:

$$d_t(x, y) = |I_t(x, y) - B(x, y)| \quad (2.10)$$

Mặt nạ được tạo bằng cách so sánh d_t với ngưỡng T. Điều kiện dùng dấu "$>$" hay "$\geq$" phải được giữ giống nhau trong chương trình tham chiếu và RTL. Ánh sáng thay đổi, bóng đổ hoặc camera rung đều có thể làm tăng độ chênh lệch mà không tương ứng với đối tượng cần tìm. Đối tượng đứng yên có thể mất khỏi mặt nạ chênh lệch hai khung.

Nhánh hộp của đồ án khai thác nền và màu trong một vùng băng tải xác định để đề xuất vùng. Không suy rộng giả thiết này sang ảnh drone có camera chuyển động. Trong công trình tham khảo [9], Zipfian Estimation được sử dụng để chọn vùng chuyển động trước HOG–SVM. Phương pháp đó có cơ chế ước lượng riêng, khác với phép chênh lệch ảnh theo nền tham chiếu đang xét.

2.5.2. Hình thái học và thành phần liên thông

Trên mặt nạ nhị phân, phép mở gồm co rồi giãn có thể loại các thành phần nhỏ; phép đóng gồm giãn rồi co có thể lấp các khoảng trống nhỏ [14]. Tiếp theo, các thành phần liên thông được xác định và chuyển thành khung bao ứng viên. Kích thước phần tử cấu trúc phải phù hợp với kích thước đối tượng, vì phép xử lý quá mạnh có thể làm mất vật nhỏ hoặc nối hai vật gần nhau.

Những quy tắc lọc theo diện tích, tỉ lệ khung hoặc vị trí phải được xác định từ dữ liệu cho phép. Nếu một hộp không tạo được ứng viên, các bước HOG và SVM phía sau không thể khôi phục nó. Vì vậy, khi phân tích lỗi cần tách lỗi tạo ứng viên, lỗi phân loại và lỗi định vị.

2.5.3. Loại khung trùng bằng NMS

NMS (Non-Maximum Suppression, loại bỏ các khung không cực đại) chọn khung có điểm cao, rồi loại các khung còn lại chồng lấn quá mức với khung đã chọn. Quá trình được lặp cho đến khi hết ứng viên hoặc đạt giới hạn số dự đoán. Độ chồng lấn giữa hai khung A và B được đo bằng IoU (Intersection over Union):

$$IoU(A,B) = \frac{|A \cap B|}{|A \cup B|} \quad (2.11)$$

Với bộ phát hiện nhiều lớp, thông thường NMS được áp dụng theo từng lớp để tránh loại dự đoán của lớp khác chỉ vì chúng giao nhau. Ngưỡng NMS là tham số hậu xử lý; ngưỡng IoU dùng ghép dự đoán với nhãn chuẩn là tham số đánh giá. Hai ngưỡng có thể khác nhau và phải được báo cáo riêng. NMS cũng có thể loại nhầm một đối tượng trong cảnh đông nếu các khung chồng lấn nhiều.

2.6. Mạng nơ-ron tích chập và YOLOv8n

2.6.1. Phép tích chập và đặc trưng học được

Khác với HOG sử dụng cách trích đặc trưng xác định trước, mạng nơ-ron tích chập học các bộ lọc từ dữ liệu. Mỗi lớp kết hợp các vùng lân cận trên bản đồ đặc trưng đầu vào để tạo bản đồ đầu ra. Với X là đầu vào, K là trọng số, b là hệ số tự do và φ là hàm kích hoạt, một dạng biểu diễn giản lược là:

$$Y_o(x,y) = \varphi\left(\sum_{c,u,v} K_{o,c}(u,v) X_c(x+u, y+v) + b_o\right) \quad (2.12)$$

Trong công thức, c là kênh đầu vào, o là kênh đầu ra, còn u và v là tọa độ trong kernel. Công thức minh họa trường hợp bước dịch bằng một; khi triển khai phải xét thêm stride, padding và kích thước tensor. Phép nhân–cộng được lặp trên nhiều vị trí và nhiều kênh nên thường chiếm phần đáng kể của khối lượng tính toán.

Đầu các lớp học những cấu trúc cục bộ; các lớp sâu kết hợp thông tin trên vùng ảnh rộng hơn. Ngoài tích chập, mạng còn có các thao tác phi tuyến, ghép kênh, thay đổi kích thước và gộp đặc trưng. Vì vậy, tăng tốc tích chập không đồng nghĩa mọi thành phần của mạng đều được tăng tốc.

2.6.2. Cấu trúc của YOLOv8n

YOLOv8n là biến thể kích thước nhỏ thuộc họ YOLOv8. Kiến trúc được xét theo ba phần: backbone trích đặc trưng, neck kết hợp đặc trưng ở nhiều độ phân giải và

head dự đoán vị trí cùng lớp. Các khối C2f tạo và ghép các nhánh đặc trưng; SPPF mở rộng thông tin ngữ cảnh bằng các phép gộp. Việc kết hợp nhiều mức đặc trưng hỗ trợ phát hiện đối tượng có kích thước khác nhau [18], [19].

Đầu phát hiện của YOLOv8 sử dụng cách biểu diễn không dựa trên hộp neo định sẵn. "Anchor-free" không có nghĩa không tồn tại điểm tham chiếu: dự đoán vị trí vẫn được giải mã tương ứng với các điểm trên bản đồ đặc trưng. Do sự khác nhau giữa các thế hệ YOLO, cách giải mã tọa độ và điểm lớp được xác định theo đầu phát hiện YOLOv8 đang sử dụng.

Trong cấu hình YOLOv8 phát hiện chuẩn, các đầu ra ở stride 8, 16 và 32 tạo thành các lưới 40 × 40, 20 × 20 và 10 × 10 khi đầu vào là 320 × 320. Tổng cộng có 2.100 vị trí dự đoán. Sau giải mã, nếu mỗi vị trí gồm 4 tọa độ và 5 điểm lớp thì tensor có 18.900 giá trị. Đây là số phần tử tensor, không phải số đối tượng được phát hiện. Cách bố trí chiều và việc đầu ra đã giải mã hay chưa phải được xác nhận theo tệp mô hình xuất thực tế.



Hình 2.1. Sơ đồ chức năng của chuỗi suy luận YOLOv8n

2.6.3. Huấn luyện, giải mã và suy luận

Trong huấn luyện, ảnh và nhãn khung bao được dùng để điều chỉnh trọng số thông qua hàm mất mát. Với YOLOv8, các thành phần liên quan đến phân loại và định vị, bao gồm biểu diễn phân bố khoảng cách ở đầu hồi quy, hỗ trợ mạng học vị trí khung [19]. Đây là quá trình diễn ra trên máy phục vụ huấn luyện; chương trình trên bo sử dụng trọng số đã xác định để suy luận.

Khi suy luận, ảnh được tiền xử lý theo đúng định dạng mô hình. Mạng tạo tensor đầu ra; chương trình giải mã nếu cần, chọn các dự đoán đủ điểm, thực hiện NMS và chuyển tọa độ về ảnh gốc. Với cấu hình năm lớp phương tiện, ánh xạ chỉ số lớp phải được giữ nhất quán giữa mô hình, nhãn chuẩn và phần hiển thị. Tên lớp giống nhau nhưng thứ tự khác nhau cũng có thể làm kết quả đánh giá sai.

Ảnh drone thường có nhiều đối tượng nhỏ và gần nhau. Co ảnh lớn về 320 × 320 có thể làm mất chi tiết của phương tiện; tăng kích thước đầu vào giữ được nhiều chi

tiết hơn nhưng tăng bộ nhớ và khối lượng tính toán. Đây là đánh đổi cần đánh giá bằng dữ liệu thực, không thể xác định chất lượng chỉ từ tên biến thể YOLOv8n.

2.6.4. Quan hệ giữa HOG–SVM và YOLO trong đồ án

Bảng 2.1. Đặc điểm của HOG–SVM và YOLOv8n

Tiêu chí	HOG–SVM	YOLOv8n
Đặc trưng	Thiết kế trước từ gradient và histogram.	Học các bộ lọc và đặc trưng từ dữ liệu.
Tìm vị trí	Cửa sổ trượt hoặc tạo vùng ứng viên rồi phân loại.	Dự đoán vị trí và lớp từ bản đồ đặc trưng của mạng.
Mô hình suy luận	Vector HOG, trọng số và hệ số SVM.	Các tensor trọng số và đồ thị các phép toán.
Khối tính toán chính	Gradient, histogram, chuẩn hóa, tích vô hướng.	Tích chập, phi tuyến, ghép và biến đổi tensor.
Điểm cần lưu ý	Phụ thuộc mạnh vào ứng viên và dữ liệu mẫu.	Phụ thuộc dữ liệu học, kích thước đầu vào và runtime.
Vai trò trong đồ án	Kế thừa bộ phát hiện hộp và mô phỏng RTL.	Nghiên cứu phương tiện trong ảnh drone và triển khai PS–PL.

Bảng 2.1 tổng hợp các đặc điểm của hai phương pháp được khảo sát trong đồ án. HOG–SVM giúp phân tích rõ chuỗi tạo ứng viên và tính đặc trưng; YOLOv8n phục vụ khảo sát phát hiện nhiều lớp và triển khai mạng nơ-ron trên SoC. Các kết luận định lượng vẫn cần gắn với đúng bộ dữ liệu, lớp đối tượng và cấu hình của từng nhánh.

2.7. Biểu diễn số và lượng tử hóa

2.7.1. Số thực, số cố định và số nguyên

Số thực dấu phẩy động có miền biểu diễn rộng nhưng việc thực thi cần phần cứng và phần mềm tương ứng. Số cố định biểu diễn một giá trị bằng số nguyên và một tỉ lệ đã quy ước. Ví dụ, với F bit phần lẻ, giá trị gần đúng là $\hat{x} = q \times 2^{-f}$. Khi cộng hai số, phải đưa chúng về cùng tỉ lệ; khi nhân, số bit phần lẻ của kết quả bằng tổng số bit phần lẻ của hai toán hạng trước khi làm tròn lại.

Lượng tử hóa affine dùng hệ số scale s > 0 và zero point z để ánh xạ giá trị thực sang miền số nguyên:

$$q = clip(round(\frac{x}{s}) + z, q_{min}, q_{max})$$

$$\hat{x} = s(q - z) \quad (2.13)$$

Hàm clip giới hạn kết quả trong miền [q_{min}, q_{max}]. Bước làm tròn và giới hạn tạo sai số; các giá trị vượt miền bị bão hòa. Lượng tử hóa đối xứng thường dùng zero point bằng 0, còn lượng tử hóa bất đối xứng cho phép dịch miền biểu diễn. Scale có thể được xác định cho cả tensor hoặc riêng từng kênh, tùy phép toán và công cụ triển khai.

2.7.2. Hiệu chuẩn và ACIQ

Lượng tử hóa sau huấn luyện sử dụng một mô hình đã có trọng số, sau đó xác định miền biểu diễn của các tensor mà không nhất thiết huấn luyện lại. Tập calibration cung cấp mẫu đầu vào để thu thập hoặc ước lượng đặc điểm phân bố. Tập này cần đại diện cho dữ liệu sử dụng, nhưng không lấy từ test dùng để báo cáo kết quả cuối cùng.

ACIQ, viết tắt của Analytical Clipping for Integer Quantization, lựa chọn ngưỡng cắt dựa trên mô hình phân bố và sai số lượng tử hóa [3]. Ngưỡng quá hẹp làm mất các giá trị lớn; ngưỡng quá rộng làm bước lượng tử lớn hơn đối với phần dữ liệu tập trung gần tâm. Cách chọn ngưỡng nhằm cân bằng hai loại sai số này. Khi sử dụng ACIQ trong một thư viện cụ thể, cần ghi phiên bản công cụ và cấu hình thực tế, vì tên phương pháp chưa mô tả đầy đủ cách áp dụng cho từng tensor.

Trong đồ án, đánh giá FP32 và INT8 cần giữ nguyên ảnh, ánh xạ lớp và quy trình hậu xử lý. Việc một bản lượng tử hóa đạt yêu cầu AP trên PC không tự chứng minh bản đó đã chạy trên bo hoặc nhanh hơn bản trước. Kết quả chất lượng và kết quả tốc độ phải được đo riêng trên đúng phiên bản mô hình.

2.7.3. Nhân tích lũy và tái lượng tử hóa

Với đầu vào và trọng số đã lượng tử hóa, phép tích vô hướng cần trừ zero point trước khi nhân. Một dạng bộ tích lũy số nguyên là:

$$a = b_{int} + \sum_{k=1}^{K} (q_{x,k} - z_x)(q_{w,k} - z_w) \quad (2.14)$$

Hệ số bias phải được đưa về đúng tỉ lệ của tích. Sau tích lũy, kết quả có thể được đổi tỉ lệ, làm tròn và giới hạn để tạo tensor tiếp theo. Cần phân biệt INT8 của tensor

lưu trữ với độ rộng phép toán nội bộ: hiệu giữa hai giá trị 8 bit có thể cần 9 bit có dấu, còn tổng nhiều tích cần bộ tích lũy rộng hơn.

Độ rộng phải được suy ra từ miền toán hạng và số phần tử cộng dồn. Tràn theo modulo và bão hòa là hai hành vi khác nhau, không được thay thế tùy ý. Các trường hợp giá trị cực trị, số âm, zero point khác 0 và nhiều tích cùng dấu được sử dụng để kiểm tra miền biểu diễn và nguy cơ tràn bộ tích lũy.

2.8. FPGA và kiến trúc SoC Zynq-7010

2.8.1. Tài nguyên logic và bộ nhớ

FPGA gồm các tài nguyên logic và kết nối có thể cấu hình. LUT thực hiện hàm logic, flip-flop lưu trạng thái, BRAM lưu dữ liệu trong chip và DSP hỗ trợ các phép tính số học. Tệp bitstream xác định cấu hình phần logic. Verilog mô tả phần cứng ở mức thanh ghi và phép chuyển dữ liệu giữa chúng; nhiều khối có thể hoạt động đồng thời, khác với cách diễn giải từng lệnh tuần tự của chương trình CPU.

Theo tài liệu Zynq-7000, Zynq-7010 có 17.600 LUT, 35.200 flip-flop, 80 DSP và 60 khối BRAM 36 Kb [1]. Đây là tài nguyên vật lý tổng cộng, không phải toàn bộ ngân sách dành cho bộ tăng tốc. Giao tiếp, bộ đệm, điều khiển và các khối hiển thị cũng sử dụng tài nguyên. Dung lượng DDR trên bo là bộ nhớ ngoài và phải được phân biệt với BRAM trong chip.

2.8.2. Hệ xử lý PS và phần logic PL

PS (Processing System) là hệ xử lý có CPU ARM Cortex-A9 và các ngoại vi; PL (Programmable Logic) là phần logic lập trình được. Chương trình trên PS có thể nạp dữ liệu, điều khiển lõi tính toán, thực hiện các lớp chưa đưa vào PL và hậu xử lý kết quả. PL thực hiện những phép tính được thiết kế thành mạch. Phân chia nhiệm vụ giữa PS và PL xác định phần tính toán được tăng tốc và phần còn thực thi bằng phần mềm [2].

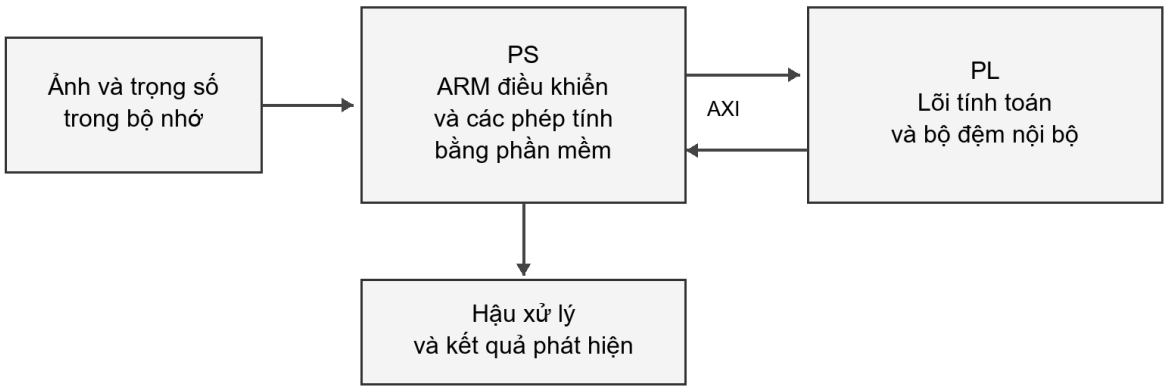
Phân chia PS–PL không chỉ dựa trên số lớp. Một số ít lớp có thể chiếm phần lớn phép nhân–cộng, hoặc ngược lại. Đánh giá cần xét số phép toán, kích thước tensor và thời gian truyền qua từng ranh giới. Nếu phần đã tăng tốc chỉ chiếm tỉ lệ nhỏ trong thời gian toàn chương trình, mức cải thiện tổng thể sẽ bị giới hạn.

2.8.3. AXI và đường truyền dữ liệu

AXI4-Lite thích hợp cho các thanh ghi điều khiển và các giao dịch đơn giản. AXI4 hỗ trợ giao dịch bộ nhớ theo cụm, còn AXI4-Stream hỗ trợ truyền luồng dữ liệu.

DMA (Direct Memory Access, truy cập bộ nhớ trực tiếp) có thể chuyển các vùng dữ liệu giữa bộ nhớ và thiết bị mà không yêu cầu CPU thực hiện từng lần đọc ghi. DMA là một phương án tổ chức truyền dữ liệu; mức độ áp dụng phụ thuộc kiến trúc giao tiếp của từng bản triển khai.

Trong thiết kế điều khiển bằng thanh ghi, phần mềm thường ghi cấu hình và dữ liệu, phát lệnh bắt đầu, chờ trạng thái hoàn tất rồi đọc kết quả. Thời gian này gồm cả trao đổi dữ liệu và đồng bộ; không thể lấy riêng số chu kỳ của lõi tính toán để suy ra tốc độ toàn mạng. Khi dùng bộ nhớ chia sẻ hoặc DMA, tính nhất quán bộ nhớ đệm cũng phải được xử lý đúng.



Hình 2.2. Nguyên lý phân chia xử lý giữa PS và PL

2.8.4. Pipeline và tái sử dụng phần cứng

Pipeline chia phép xử lý thành nhiều tầng có thanh ghi trung gian. Sau giai đoạn nạp đầy, nhiều phần dữ liệu có thể được xử lý đồng thời ở các tầng khác nhau. Độ trễ là thời gian một dữ liệu đi qua các tầng; khoảng cách khởi tạo II là số chu kỳ tối thiểu giữa hai dữ liệu được nhận. Pipeline có nhiều tầng vẫn có thể đạt thông lượng cao nếu II nhỏ.

Tái sử dụng một lõi cho nhiều lớp hoặc nhiều phần tensor giúp giảm tài nguyên nhưng cần bộ đệm và điều khiển lịch thực thi. Ngược lại, cấp phần cứng riêng cho nhiều tầng có thể giảm trao đổi trung gian nhưng dễ vượt tài nguyên. Đối với Zynq-7010, lựa chọn này phải dựa trên báo cáo tổng hợp, triển khai và kết quả đo thực tế.

2.9. Mô phỏng và kiểm chứng phần cứng

2.9.1. Testbench và dữ liệu tham chiếu

RTL (Register Transfer Level) là mức mô tả hoạt động phần cứng thông qua thanh ghi và luồng dữ liệu giữa chúng. Testbench là chương trình mô phỏng tạo clock,

reset, kích thích đầu vào và kiểm tra đầu ra của mạch. Testbench có thể đọc dữ liệu ảnh từ tệp và ghi ảnh kết quả, nhưng các thao tác đọc ghi tệp này không tự trở thành phần cứng. Trên bo thật, dữ liệu cần được cung cấp bằng bộ nhớ hoặc giao tiếp được thiết kế cụ thể.

Một testbench tự kiểm tra so sánh kết quả RTL với mô hình tham chiếu và ghi số sai khác. Với số nguyên có quy tắc hoàn toàn giống nhau có thể yêu cầu khớp từng bit; với phép xấp xỉ hoặc lượng tử hóa cần xác định dung sai. Các mẫu kiểm thử phải bao gồm trường hợp thông thường, dữ liệu cực trị, reset giữa luồng và thời điểm ngắt quãng dữ liệu.

2.9.2. Tín hiệu hợp lệ và cách đọc dạng sóng

Tín hiệu valid cho biết dữ liệu đang có ý nghĩa; tín hiệu ready cho biết khối nhận có thể tiếp nhận. Trong giao tiếp dùng cả hai, dữ liệu được chấp nhận tại cạnh clock khi valid và ready cùng bằng 1. Khi đầu ra không hợp lệ, giá trị còn giữ trên bus có thể là kết quả cũ và không được đếm là một phát hiện mới.

Các tín hiệu như image_start, busy, detection_valid và image_done giúp kiểm tra trình tự xử lý ảnh. Khi detection_valid được kích hoạt, tọa độ và điểm tương ứng phải ổn định theo giao thức. Bộ đếm errors tổng hợp các sai khác mà testbench kiểm tra. Kết quả này đánh giá tính đúng của mô phỏng theo bộ mẫu, còn recall và AP được xác định riêng bằng nhãn đối tượng.

2.9.3. Từ mô phỏng đến chạy trên bo

Mô phỏng hành vi kiểm tra logic của mô tả RTL. Tổng hợp chuyển RTL thành mạng phần tử; triển khai thực hiện đặt và định tuyến trên thiết bị mục tiêu. Báo cáo tài nguyên và timing cho biết thiết kế có vừa chip và đáp ứng ràng buộc thời gian hay không. Việc mô phỏng thành công chưa đủ để kết luận thiết kế đã tổng hợp được, đạt timing hoặc hoạt động với thiết bị ngoại vi thật.

JTAG phục vụ cấu hình và kiểm tra thiết bị. UART phục vụ truyền thông nối tiếp, thường dùng để ghi thông báo, kết quả và thời gian đo. Chương trình chạy trực tiếp trên ARM không dùng hệ điều hành được gọi là bare-metal. PetaLinux là bộ công cụ xây dựng hệ Linux nhúng, cung cấp một hướng tổ chức phần mềm có hệ thống tệp, mạng và trình điều khiển ngoại vi. Bare-metal và Linux là hai lựa chọn môi trường thực thi; cấu hình sử dụng được xác định trong từng thí nghiệm.

2.10. Edge AI và yêu cầu thời gian thực

2.10.1. Phân biệt xử lý tại biên và tốc độ xử lý

Một hệ thống thuộc hướng Edge AI khi thực hiện suy luận tại thiết bị gần nguồn dữ liệu, thay vì yêu cầu máy chủ từ xa xử lý mọi đầu vào. Huấn luyện trên PC rồi chuyển trọng số sang bo để suy luận vẫn phù hợp với cách tổ chức này. Tuy nhiên, Edge AI không đồng nghĩa với việc toàn bộ mạng phải chạy trên PL, cũng không bảo đảm tốc độ thời gian thực.

Thời gian thực phụ thuộc hạn đáp ứng của ứng dụng. Đối với robot, cần xét thời gian từ lúc thu nhận cảnh đến khi kết quả có thể được khối điều khiển sử dụng. Một giá trị FPS trung bình cao vẫn có thể đi kèm các khung bị trễ lớn. Vì vậy, ngoài trung bình cần xem phân bố thời gian và các giá trị phần trăm như P95 khi có đủ số lượt đo.

2.10.2. Độ trễ, thông lượng và giới hạn tăng tốc

Với N ảnh được xử lý trong tổng thời gian T, thông lượng đo được là:

$$FPS = \frac{N}{T} \quad (2.15)$$

Với một chuỗi thực thi nối tiếp không chồng lấn, thời gian trung bình mỗi ảnh là T/N. Với pipeline chồng lấn, thông lượng và độ trễ của một ảnh cần được đo riêng. Thời gian đầu cuối có thể gồm thu nhận, tiền xử lý, truyền dữ liệu, suy luận, hậu xử lý và hiển thị; các khâu chồng lấn không được cộng máy móc khi tính thông lượng.

Theo mô hình Amdahl giản lược, nếu tỉ lệ thời gian p được tăng tốc A lần thì mức tăng tốc toàn chương trình là:

$$S = \frac{1}{(1 - p) + \frac{p}{A}} \quad (2.16)$$

Biểu thức giả thiết không phát sinh chi phí mới và phần còn lại giữ nguyên. Trong hệ PS–PL, chi phí truyền tensor và đồng bộ có thể làm kết quả thực tế thấp hơn ước lượng. Đây là lý do cần đo thời gian từng phần trước khi quyết định tăng số bộ nhân hoặc thay đổi kiến trúc.

2.11. Phương pháp đánh giá kết quả

2.11.1. Ghép dự đoán với nhãn chuẩn

Tại một ngưỡng điểm và ngưỡng IoU đã chọn, dự đoán được ghép với nhãn chuẩn cùng lớp theo quy tắc một–một. Dự đoán ghép đúng là TP, dự đoán không ghép

được là FP, còn nhãn chuẩn không có dự đoán phù hợp là FN. Nhiều khung trùng quanh cùng một đối tượng không được cùng tính là TP. Cách xử lý vùng bỏ qua và đối tượng bị che khuất phải theo giao thức của tập dữ liệu hoặc được khai báo rõ nếu dùng giao thức rút gọn.

Các chỉ tiêu precision, recall và F1 được xác định bởi:

$$Precision = \frac{TP}{TP + FP}; \quad Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \qquad (2.17)$$

Precision phản ánh tỉ lệ phát hiện đúng trong các dự đoán, còn recall phản ánh tỉ lệ đối tượng chuẩn được tìm thấy. F1 tổng hợp hai chỉ tiêu tại cùng một điểm làm việc. Khi mẫu số bằng 0, chương trình đánh giá cần có quy ước nhất quán và không tự động coi trường hợp không có đối tượng là kết quả phát hiện hoàn hảo.

2.11.2. AP và mAP

Khi thay đổi ngưỡng điểm, precision và recall tạo thành đường cong P–R. AP tổng hợp đường cong theo một quy tắc nội suy; một dạng dùng 101 mức recall là:

$$AP = \frac{1}{101} \sum_{j=0}^{100} P_{interp}(\frac{j}{100}) \qquad (2.18)$$

Trong đó P_interp(r) là precision nội suy tại mức recall r. AP50 dùng ngưỡng IoU 0,50; AP@[0,50:0,95] lấy trung bình AP ở các ngưỡng từ 0,50 đến 0,95 với bước 0,05. mAP là trung bình theo các lớp được đánh giá. Khi chỉ có một lớp, mAP bằng AP của lớp đó tại cùng quy tắc [7].

Precision và recall có thể cùng đạt 100% ở IoU 0,50 trong khi AP@[0,50:0,95] vẫn thấp hơn vì khung chưa đủ sát tại các ngưỡng IoU cao hơn. AP trên toàn tập cũng không nhất thiết bằng trung bình các AP tính riêng cho từng ảnh, vì việc xếp hạng dự đoán được thực hiện trên tập các ảnh theo giao thức đánh giá.

2.11.3. Tách dữ liệu và khả năng tổng quát hóa

Ảnh train phục vụ học tham số, validation phục vụ chọn cấu hình và test phục vụ đánh giá cấu hình đã cố định. Các ảnh hoặc vùng cắt từ cùng một khung gốc phải được quản lý để tránh xuất hiện ở các tập khác nhau. Với dữ liệu video, các khung gần nhau có thể rất giống nhau; tách theo video hoặc phiên thu nhận thường cung cấp phép kiểm tra độc lập hơn so với chia xen kẽ khung hình.

Ảnh ngoài dataset giúp khảo sát sự thay đổi phân bố về nền, ánh sáng, góc nhìn và kích thước đối tượng. Nếu chưa có nhãn chuẩn, chỉ có thể nhận xét định tính về khung vẽ; không đủ cơ sở tính precision, recall hay AP. Khi tạo biến thể ảnh bằng biến đổi số, cần ghi rõ nguồn và quan hệ với ảnh gốc, không tính chúng như những cảnh thu nhận độc lập.

2.11.4. Đánh giá mô phỏng và phần cứng

Bảng 2.2. Phân biệt các mức bằng chứng thực nghiệm

Mức kiểm tra	**Có thể xác nhận**	**Chưa đủ để xác nhận**
Dạng sóng RTL	Trình tự và thời điểm tín hiệu trong mô hình.	Thiết kế đạt timing hoặc đã chạy trên bo.
Đối chiếu đầu ra số học	Tính đúng với mô hình tham chiếu và dung sai đã đặt.	Độ chính xác phát hiện trên mọi ảnh.
Đánh giá ảnh có nhãn	Precision, recall, AP trên tập ảnh đã đánh giá.	Khả năng tổng quát sang mọi cảnh hoặc camera.
Tổng hợp và triển khai	Tài nguyên và timing theo ràng buộc thiết kế.	Hiệu năng đầu cuối với giao tiếp thật.
Chạy và đo trên bo	Hành vi và tốc độ ở cấu hình thí nghiệm.	Tốc độ camera, video hoặc bản mô hình khác chưa đo.

So sánh giữa PC và RTL cần dùng cùng đầu vào và nêu rõ khác biệt thuật toán, kiểu số, ngưỡng và tiền xử lý. So sánh tốc độ cần nêu phần thời gian được tính, số lượt làm nóng, số lượt đo và cách tổng hợp. Thời gian chạy bộ mô phỏng trên máy tính không phải thời gian xử lý của FPGA vật lý. Tần số làm mới LCD và tốc độ suy luận được đo riêng vì một kết quả phát hiện có thể được hiển thị lặp lại nhiều lần.

2.12. Kết luận chương

Chương 2 đã trình bày cơ sở để xây dựng và đánh giá hai nhánh phát hiện đối tượng. HOG–SVM dựa trên đặc trưng gradient và bộ phân loại tuyến tính; YOLOv8n học đặc trưng và dự đoán khung bao bằng mạng nơ-ron. Khi triển khai trên SoC FPGA, cả hai đều cần chú ý biểu diễn số, bộ nhớ, giao tiếp và tổ chức tính toán. Những nội dung này là căn cứ cho thiết kế hệ thống ở Chương 3 và kiểm chứng chất lượng, tốc độ, tài nguyên ở Chương 4.

TÀI LIỆU THAM KHẢO

Tiếng Anh

[1] AMD, *Zynq-7000 SoC Data Sheet: Overview*, DS190.

[2] AMD, *Zynq-7000 SoC Technical Reference Manual*, UG585.

[3] R. Banner et al., "Post-training 4-bit Quantization of Convolution Networks for Rapid-deployment", arXiv:1810.05723, 2018, revised 2019.

[4] M. Blott et al., "FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks", *ACM Transactions on Reconfigurable Technology and Systems*, Vol. 11, No. 3, 2018. arXiv:1809.04570.

[5] C. Cortes and V. Vapnik, "Support-Vector Networks", *Machine Learning*, Vol. 20, 1995, pp. 273–297. DOI: 10.1007/BF00994018.

[6] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection", *Proceedings of CVPR*, Vol. 1, 2005, pp. 886–893.

[7] P. Dollár and T.-Y. Lin, "cocoeval.py", *COCO Evaluation API*, cocodataset/cocoapi source code repository.

[8] M. Hahnle, F. Saxen, M. Hisung, U. Brunsmann and K. Doll, "FPGA-Based Real-Time Pedestrian Detection on High-Resolution Images", *Proceedings of CVPR Workshops*, 2013. DOI: 10.1109/CVPRW.2013.95.

[9] Nguyen Trung Kien, *Developing a Real-Time Object Detection System on FPGA*, Master thesis, Université Paris-Saclay, 2024, pp. 1–23.

[10] J. Luo and C. H. Lin, "Pure FPGA Implementation of an HOG Based Real-Time Pedestrian Detection System", *Sensors*, Vol. 18, No. 4, Article 1174, 2018. DOI: 10.3390/s18041174.

[11] K. Mizuno, Y. Terachi, K. Takagi, S. Izumi, H. Kawaguchi and M. Yoshimoto, "Architectural Study of HOG Feature Extraction Processor for Real-Time Object Detection", *Proceedings of SiPS*, 2012, pp. 197–202.

[12] OpenCV, "Color Conversions", *OpenCV Documentation*, accessed November 9, 2026.

[13] OpenCV, "cv::HOGDescriptor Struct Reference", *OpenCV Documentation*, accessed November 9, 2026.

[14] OpenCV, “Morphological Transformations”, *OpenCV Documentation*, accessed November 9, 2026.

[15] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection”, *Proceedings of CVPR*, 2016, pp. 779–788.

[16] S. Somton, *Cement Conveyor Box Detection*, Roboflow Universe, Version 1.

[17] A. Suleiman and V. Sze, “Energy-Efficient HOG-Based Object Detection at 1080HD 60 fps with Multi-Scale Support”, *Proceedings of SiPS*, 2014.

[18] Ultralytics, “Explore Ultralytics YOLOv8”, *Ultralytics Documentation*, accessed November 9, 2026.

[19] Ultralytics, “yolov8.yaml; Detect; DFL; loss”, ultralytics/ultralytics source code repository, accessed November 9, 2026.

[20] Y. Umuroglu et al., “FINN: A Framework for Fast, Scalable Binarized Neural Network Inference”, *Proceedings of ACM/SIGDA FPGA*, 2017. DOI: 10.1145/3020078.3021744.

[21] VisDrone, *VisDrone Dataset*, official dataset repository and documentation.

[22] M. Wasala and T. Kryjak, “Real-time HOG+SVM Based Object Detection Using SoC FPGA for a UHD Video Stream”, arXiv:2204.10619, 2022.